TALLER PRÁCTICO

Registro y autenticación de usuarios con Spring Boot, Spring Security y MySQL

Java 25 · Spring Boot 3.5.x · Spring Security 6.x · Spring Data JPA · Thymeleaf

Asignatura: Desarrollo Web Avanzado
Modalidad: Taller guiado e incremental
Producto: Aplicación web funcional con registro, login, logout y recursos protegidos

Nota: *Guía adaptada pedagógicamente a partir del tutorial de CodeJava "Spring Boot Registration and Login with MySQL Database Tutorial", modernizando la configuración para el ecosistema Spring Boot 3.x / Spring Security 6.x y Java 25.*

1. Contextualización

La mayoría de las aplicaciones web necesitan identificar usuarios, validar credenciales y decidir qué recursos puede consultar cada persona. En este taller se construirá una aplicación MVC que registra usuarios en MySQL, almacena las contraseñas mediante BCrypt, autentica por correo electrónico y protege el listado de usuarios con Spring Security.

El desarrollo se realizará de forma incremental. Cada etapa incluye un punto de verificación. El estudiante no debe avanzar a la etapa siguiente si el resultado esperado de la etapa actual no funciona.

2. Resultados de aprendizaje

- Configurar un proyecto Maven con Spring Boot y Java 25.
- Persistir entidades con Spring Data JPA e Hibernate en MySQL.
- Implementar formularios y vistas con Spring MVC y Thymeleaf.
- Diferenciar identificación, autenticación y autorización.
- Implementar autenticación basada en UserDetailsService y DaoAuthenticationProvider.
- Codificar contraseñas con BCryptPasswordEncoder.
- Configurar reglas de acceso con SecurityFilterChain.
- Validar el funcionamiento mediante pruebas de repositorio y pruebas funcionales.

3. Arquitectura de la solución

```
Navegador
   │
   ▼
Spring Security Filter Chain
   │
   ▼
Controller (Spring MVC)
   │
   ▼
Repository (Spring Data JPA)
   │
   ▼
Hibernate / JDBC
   │
   ▼
MySQL
```

Durante el login, el flujo principal será:

```
Formulario de login
      │
      ▼
Spring Security
      │
      ▼
DaoAuthenticationProvider
      ├──► CustomUserDetailsService ──► UserRepository ──► MySQL
      └──► PasswordEncoder (BCrypt)
      │
      ▼
SecurityContext → acceso autorizado
```

4. Prerrequisitos

- JDK 25 instalado y configurado.
- IDE: IntelliJ IDEA, Spring Tool Suite o equivalente.

- Maven 3.9+ o Maven Wrapper incluido en el proyecto.
- MySQL Server y MySQL Workbench.
- Conocimientos básicos de Java, POO, HTML, SQL, MVC y Maven.

> ***Nota:*** *Para mantener el taller dentro de la línea Spring Boot 3.x, se propone Spring Boot 3.5.16. Spring gestiona las versiones compatibles de sus dependencias; no agregue versiones manuales a los starters salvo que exista una razón técnica.*

5. Etapa 1 — Crear el proyecto

Cree un proyecto Maven desde Spring Initializr con los siguientes datos:

Propiedad	Valor sugerido
Project	Maven
Language	Java
Spring Boot	3.5.16
Group	com.taller
Artifact	registro-login
Name	registro-login
Package	com.taller.registrologin
Packaging	Jar
Java	25

Dependencias iniciales:

- Spring Web
- Thymeleaf
- Spring Data JPA
- MySQL Driver
- Spring Security
- Spring Boot DevTools
- Spring Boot Starter Test

5.1 pom.xml completo

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.16</version>
        <relativePath/>
    </parent>

    <groupId>com.taller</groupId>
    <artifactId>registro-login</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>registro-login</name>

    <properties>
        <java.version>25</java.version>
    </properties>

    <dependencies>
        <dependency>
```

```
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>

        <dependency>
            <groupId>org.thymeleaf.extras</groupId>
            <artifactId>thymeleaf-extras-springsecurity6</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

PUNTO DE VERIFICACIÓN — Proyecto base

- **El IDE reconoce el proyecto Maven sin dependencias en rojo.**
- **java -version muestra JDK 25.**
- **mvn -v (o ./mvnw -v) muestra Java 25.**
- **La clase principal de Spring Boot puede iniciar, aunque todavía no exista la base de datos configurada.**

6. Etapa 2 — Crear y configurar MySQL

Desde MySQL Workbench ejecute:

```
CREATE DATABASE registro_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

```
USE registro_db;
```

Configure src/main/resources/application.properties:

```
spring.application.name=registro-login

spring.datasource.url=jdbc:mysql://localhost:3306/registro_db?useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=SU_PASSWORD

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.thymeleaf.cache=false
```

Nota: *Reemplace SU_PASSWORD. En un proyecto real no se recomienda versionar credenciales; posteriormente puede externalizarlas mediante variables de entorno.*

Discusión: el tutorial original usa inicialmente ddl-auto=create para que Hibernate genere la tabla durante la prueba. En este taller usaremos update para evitar que los datos se destruyan al reiniciar mientras se aprende. En producción se prefieren migraciones controladas (por ejemplo, Flyway o Liquibase).

PUNTO DE VERIFICACIÓN — Conexión a MySQL

- **La base registro_db existe.**
- **La aplicación inicia sin errores de conexión JDBC.**
- **No aparece Access denied for user ni Communications link failure.**

7. Etapa 3 — Crear la entidad User

Cree model/User.java:

```
package com.taller.registrologin.model;

import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @Column(nullable = false, length = 100)
    private String password;

    @Column(name = "first_name", nullable = false, length = 50)
    private String firstName;

    @Column(name = "last_name", nullable = false, length = 50)
    private String lastName;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getEmail() {
        return email;
    }
```

```
    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}
```

Analice las anotaciones @Entity, @Table, @Id, @GeneratedValue y @Column. Observe que Spring Boot 3 utiliza jakarta.persistence y no javax.persistence.

8. Etapa 4 — Repositorio JPA

```
package com.taller.registrologin.repository;

import com.taller.registrologin.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByEmail(String email);

    boolean existsByEmail(String email);
}
```

Spring Data deriva las consultas a partir de los nombres findByEmail y existsByEmail. No es necesario escribir SQL ni JPQL para estos casos.

8.1 Prueba del repositorio

Cree src/test/java/com/taller/registrologin/repository/UserRepositoryTests.java:

```
package com.taller.registrologin.repository;

import com.taller.registrologin.model.User;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.jdbc.AutoConfigureTestDatabase;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;

import static org.assertj.core.api.Assertions.assertThat;

@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
class UserRepositoryTests {
```

```
    @Autowired
    private UserRepository userRepository;

    @Test
    void debeGuardarYBuscarUsuarioPorEmail() {
        User user = new User();
        user.setEmail("estudiante.prueba@correo.com");
        user.setPassword("temporal");
        user.setFirstName("Estudiante");
        user.setLastName("Prueba");

        User saved = userRepository.saveAndFlush(user);

        assertThat(saved.getId()).isNotNull();
        assertThat(userRepository.findByEmail("estudiante.prueba@correo.com"))
                .isPresent();
    }
}
```

Nota: *Esta prueba utiliza la base MySQL real porque Replace.NONE impide sustituirla por una base embebida. La transacción de @DataJpaTest se revierte por defecto al finalizar la prueba; esto es conveniente para no contaminar la base.*

PUNTO DE VERIFICACIÓN — Persistencia

- **La prueba JPA termina en verde.**
- **Hibernate genera o actualiza la tabla users.**
- **DESCRIBE users muestra id, email, password, first_name y last_name.**
- **El campo email posee una restricción de unicidad.**

9. Etapa 5 — Servicio de registro

Separaremos la lógica de negocio del controlador. Cree service/UserService.java:

```
package com.taller.registrologin.service;

import com.taller.registrologin.model.User;
import com.taller.registrologin.repository.UserRepository;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public UserService(UserRepository userRepository,
                       PasswordEncoder passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    public User register(User user) {
        if (userRepository.existsByEmail(user.getEmail())) {
            throw new IllegalArgumentException(
                    "El correo electrónico ya se encuentra registrado.");
        }

        user.setPassword(passwordEncoder.encode(user.getPassword()));
        return userRepository.save(user);
    }

    public List<User> findAll() {
        return userRepository.findAll();
```

```
    }
}
```

Nota: *El PasswordEncoder se declarará como Bean en SecurityConfig. Spring podrá inyectarlo en UserService.*

10. Etapa 6 — Configuración de Spring Security

Antes del controlador, configure el encoder y las reglas de acceso. Cree config/SecurityConfig.java:

```
package com.taller.registrologin.config;

import com.taller.registrologin.security.CustomUserDetailsService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    DaoAuthenticationProvider authenticationProvider(
            CustomUserDetailsService userDetailsService,
            PasswordEncoder passwordEncoder) {

        DaoAuthenticationProvider provider =
                new DaoAuthenticationProvider(userDetailsService);

        provider.setPasswordEncoder(passwordEncoder);
        return provider;
    }

    @Bean
    SecurityFilterChain securityFilterChain(
            HttpSecurity http,
            DaoAuthenticationProvider authenticationProvider)
            throws Exception {

        http
            .authenticationProvider(authenticationProvider)
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(
                    "/",
                    "/register",
                    "/process_register",
                    "/css/**",
                    "/js/**"
                ).permitAll()
                .requestMatchers("/users").authenticated()
                .anyRequest().permitAll()
            )
            .formLogin(form -> form
                .usernameParameter("email")
                .defaultSuccessUrl("/users", true)
                .permitAll()
            )
            .logout(logout -> logout
                .logoutSuccessUrl("/")
                .permitAll()
            );

        return http.build();
    }
}
```

¿Por qué se usa el constructor DaoAuthenticationProvider(userDetailsService)? En versiones actuales de Spring Security el proveedor recibe explícitamente el UserDetailsService en su constructor; después se configura el PasswordEncoder.

Todavía faltan CustomUserDetailsService y CustomUserDetails; por ello el proyecto no compilará hasta completar la siguiente etapa.

11. Etapa 7 — Adaptar el usuario a Spring Security

11.1 CustomUserDetails

```
package com.taller.registrologin.security;

import com.taller.registrologin.model.User;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.util.Collection;
import java.util.List;

public class CustomUserDetails implements UserDetails {

    private final User user;

    public CustomUserDetails(User user) {
        this.user = user;
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of();
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }

    @Override
    public String getUsername() {
        return user.getEmail();
    }

    public String getFullName() {
        return user.getFirstName() + " " + user.getLastName();
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return true;
    }
}
```

11.2 CustomUserDetailsService

```
package com.taller.registrologin.security;

import com.taller.registrologin.model.User;
import com.taller.registrologin.repository.UserRepository;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String email)
            throws UsernameNotFoundException {

        User user = userRepository.findByEmail(email)
                .orElseThrow(() ->
                    new UsernameNotFoundException("Usuario no encontrado"));

        return new CustomUserDetails(user);
    }
}
```

Spring Security utilizará el correo como username lógico. DaoAuthenticationProvider pedirá el UserDetails a este servicio y verificará la contraseña mediante el PasswordEncoder.

PUNTO DE VERIFICACIÓN — Infraestructura de seguridad

- **El proyecto compila.**
- **No se usa WebSecurityConfigurerAdapter ni antMatchers.**
- **CustomUserDetailsService recupera el usuario por correo.**
- **El PasswordEncoder es BCryptPasswordEncoder.**

12. Etapa 8 — Controlador MVC

```
package com.taller.registrologin.controller;

import com.taller.registrologin.model.User;
import com.taller.registrologin.service.UserService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class AppController {

    private final UserService userService;

    public AppController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/")
    public String viewHomePage() {
        return "index";
    }

    @GetMapping("/register")
```

```
    public String showRegistrationForm(Model model) {
        model.addAttribute("user", new User());
        return "signup_form";
    }

    @PostMapping("/process_register")
    public String processRegister(User user, Model model) {
        try {
            userService.register(user);
            return "register_success";
        } catch (IllegalArgumentException ex) {
            model.addAttribute("error", ex.getMessage());
            return "signup_form";
        }
    }

    @GetMapping("/users")
    public String listUsers(Model model) {
        model.addAttribute("listUsers", userService.findAll());
        return "users";
    }
}
```

13. Etapa 9 — Crear las vistas Thymeleaf

Las vistas se ubican en src/main/resources/templates. Para reducir dependencias externas, este taller usa CSS propio en lugar de WebJars. Cree también src/main/resources/static/css/app.css.

13.1 app.css

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
    background: #f4f6f8;
    color: #222;
}

.container {
    max-width: 900px;
    margin: 40px auto;
    background: white;
    padding: 30px;
    border-radius: 10px;
}

nav a {
    margin-right: 15px;
}

form {
    max-width: 560px;
}

.form-group {
    margin-bottom: 16px;
}

label {
    display: block;
    margin-bottom: 6px;
    font-weight: bold;
}

input {
    width: 100%;
    padding: 10px;
    box-sizing: border-box;
}

button, .button {
```

```
    display: inline-block;
    padding: 10px 16px;
    border: 0;
    border-radius: 6px;
    cursor: pointer;
    text-decoration: none;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
}

th, td {
    border: 1px solid #ccc;
    padding: 10px;
    text-align: left;
}

.error {
    padding: 10px;
    margin-bottom: 15px;
    background: #ffe6e6;
}
```

13.2 index.html

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Inicio</title>
    <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
<div class="container">
    <h1>Sistema de usuarios</h1>

    <p>
        Aplicación demostrativa de registro y autenticación
        con Spring Boot y Spring Security.
    </p>

    <nav>
        <a th:href="@{/register}">Registrar usuario</a>
        <a th:href="@{/login}">Iniciar sesión</a>
        <a th:href="@{/users}">Lista de usuarios</a>
    </nav>
</div>
</body>
</html>
```

13.3 signup_form.html

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Registro</title>
    <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
<div class="container">
    <h1>Registro de usuario</h1>

    <div class="error" th:if="${error}" th:text="${error}"></div>

    <form th:action="@{/process_register}"
          th:object="${user}"
          method="post">
```

```
        <div class="form-group">
            <label for="email">Correo electrónico</label>
            <input id="email" type="email"
                   th:field="*{email}" required maxlength="100">
        </div>

        <div class="form-group">
            <label for="password">Contraseña</label>
            <input id="password" type="password"
                   th:field="*{password}"
                   required minlength="8" maxlength="72">
        </div>

        <div class="form-group">
            <label for="firstName">Nombre</label>
            <input id="firstName" type="text"
                   th:field="*{firstName}"
                   required minlength="2" maxlength="50">
        </div>

        <div class="form-group">
            <label for="lastName">Apellido</label>
            <input id="lastName" type="text"
                   th:field="*{lastName}"
                   required minlength="2" maxlength="50">
        </div>

        <button type="submit">Registrar</button>
        <a th:href="@{/}">Volver</a>
    </form>
</div>
</body>
</html>
```

13.4 register_success.html

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Registro exitoso</title>
    <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
<div class="container">
    <h1>Registro exitoso</h1>
    <p>El usuario fue registrado correctamente.</p>
    <a class="button" th:href="@{/login}">Iniciar sesión</a>
    <a th:href="@{/}">Inicio</a>
</div>
</body>
</html>
```

13.5 users.html

```
<!DOCTYPE html>
<html lang="es"
      xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="https://www.thymeleaf.org/thymeleaf-extras-springsecurity6">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Usuarios</title>
    <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
<div class="container">

    <div>
        Bienvenido,
        <strong>
```

```
            <span sec:authentication="principal.fullName"></span>
        </strong>
    </div>

    <form th:action="@{/logout}" method="post">
        <button type="submit">Cerrar sesión</button>
    </form>

    <h1>Usuarios registrados</h1>

    <table>
        <thead>
        <tr>
            <th>ID</th>
            <th>Correo</th>
            <th>Nombre</th>
            <th>Apellido</th>
        </tr>
        </thead>
        <tbody>
        <tr th:each="user : ${listUsers}">
            <td th:text="${user.id}"></td>
            <td th:text="${user.email}"></td>
            <td th:text="${user.firstName}"></td>
            <td th:text="${user.lastName}"></td>
        </tr>
        </tbody>
    </table>
</div>
</body>
</html>
```

Nota: *No creamos login.html todavía. Spring Security proporciona una página de login por defecto. Como usernameParameter se configuró como email, el campo se procesa como correo electrónico.*

PUNTO DE VERIFICACIÓN — Registro web

- **http://localhost:8080 abre la página de inicio.**
- **/register muestra el formulario.**
- **Un usuario nuevo se almacena en MySQL.**
- **SELECT id,email,password,first_name,last_name FROM users muestra un hash BCrypt, no la contraseña original.**
- **Intentar registrar el mismo correo muestra un mensaje de error.**

14. Etapa 10 — Prueba integral de autenticación

1. Inicie la aplicación.
2. Abra http://localhost:8080.
3. Registre un usuario con una contraseña de al menos 8 caracteres.
4. Consulte la tabla users en MySQL y confirme que password no contiene el texto original.
5. Cierre cualquier sesión activa o utilice una ventana privada.
6. Escriba directamente http://localhost:8080/users.
7. Compruebe que Spring Security redirige al login.
8. Ingrese el correo y la contraseña del usuario registrado.
9. Compruebe la redirección a /users.
10. Verifique que aparece el nombre completo del usuario autenticado.
11. Pulse Cerrar sesión.
12. Intente nuevamente acceder a /users y confirme que vuelve a solicitar autenticación.

PUNTO DE VERIFICACIÓN — Aplicación funcional

- **Registro correcto.**
- **Contraseña BCrypt.**

- **Login correcto con email.**
- **/users protegido.**
- **Usuario autenticado visible.**
- **Logout funcional.**
- **Acceso posterior a /users vuelve a exigir autenticación.**

15. Consultas SQL de apoyo

```
USE registro_db;

SHOW TABLES;

DESCRIBE users;

SELECT id, email, password, first_name, last_name
FROM users
ORDER BY id;
```

Nunca modifique manualmente el hash de password para intentar asignar una contraseña en texto plano; Spring Security comparará la contraseña presentada con el hash mediante PasswordEncoder.

16. Explicación conceptual

Concepto	Aplicación en el taller
Identificación	El usuario declara quién es. En este taller lo hace mediante su correo electrónico.
Autenticación	El sistema comprueba que la identidad declarada es válida, verificando la contraseña.
Autorización	Una vez autenticado, el sistema determina qué recursos puede consultar el usuario.
UserDetails	Representación del usuario que Spring Security utiliza durante la autenticación.
UserDetailsService	Estrategia que localiza los datos del usuario. Aquí consulta UserRepository por email.
DaoAuthenticationProvider	AuthenticationProvider que combina UserDetailsService y PasswordEncoder para validar usuario y contraseña.
PasswordEncoder	Componente que transforma y verifica contraseñas sin necesitar recuperar el texto original.
SecurityFilterChain	Cadena de filtros y reglas de seguridad aplicada a las solicitudes HTTP.

17. Errores frecuentes y solución

Síntoma	Qué revisar
Access denied for user 'root'@'localhost'	Usuario o contraseña de MySQL incorrectos. Revise application.properties y pruebe las credenciales en Workbench.
Communications link failure	MySQL no está iniciado, puerto incorrecto o URL JDBC errónea.
Table 'users' doesn't exist	Compruebe que la entidad está dentro del paquete escaneado y que ddl-auto permite crear/actualizar el esquema.

Not a managed type: User	La clase User no tiene @Entity o está fuera del árbol de paquetes de la clase principal.
Template ... might not exist	El HTML no está en src/main/resources/templates o el nombre retornado por el controlador no coincide.
There is no PasswordEncoder mapped for id 'null'	La contraseña existente no está codificada como espera la configuración o se mezclaron esquemas de PasswordEncoder.
Bad credentials	Correo inexistente, contraseña incorrecta o registro creado previamente en texto plano.
Duplicate entry ... for key email	Ya existe el correo. El servicio debe comprobar existsByEmail antes de guardar.
403 al enviar un formulario POST	Spring Security protege POST con CSRF. En formularios Thymeleaf con th:action, el token se integra automáticamente; no desactive CSRF para 'resolver' el problema.
NoSuchMethodError / incompatibilidad	Evite fijar versiones arbitrarias de Spring Security; permita que Spring Boot gestione el BOM compatible.

18. Preguntas de análisis

13. Explique la diferencia entre autenticación y autorización usando un ejemplo de esta aplicación.
14. ¿Por qué email puede actuar como username para Spring Security?
15. ¿Qué responsabilidad tiene CustomUserDetailsService?
16. ¿Qué responsabilidad tiene CustomUserDetails?
17. ¿Por qué no se debe almacenar una contraseña en texto plano?
18. ¿Qué papel desempeña BCryptPasswordEncoder durante el registro y durante el login?
19. ¿Qué significa permitAll()?
20. ¿Qué significa authenticated()?
21. ¿Por qué /users redirige al login cuando el usuario no está autenticado?
22. ¿Cómo genera Spring Data JPA la implementación de findByEmail()?
23. ¿Qué ventaja tiene Optional<User> frente a retornar User y usar null?
24. ¿Por qué la columna email debe ser unique?
25. ¿Qué riesgo existe al usar ddl-auto=create después de tener datos importantes?
26. ¿Por qué no es recomendable desactivar CSRF globalmente en una aplicación web con sesiones?
27. ¿Qué ventaja aporta la inyección por constructor frente a @Autowired sobre atributos?

19. Reto de ampliación A — Login personalizado

Reemplace la página de login generada por Spring Security por una vista propia. Investigue formLogin(form -> form.loginPage("/login")...) y cree un endpoint GET /login y login.html. Mantenga el nombre del campo de correo consistente con usernameParameter("email").

Criterios mínimos: mensajes de credenciales inválidas, mensaje posterior al logout, diseño coherente con la aplicación y protección CSRF del POST.

20. Reto de ampliación B — Roles USER y ADMIN

Amplíe el modelo para diferenciar usuarios normales y administradores. El objetivo funcional será:

```
/                → público
/register        → público
/login           → público
```

```
/profile          → USER o ADMIN
/users            → ADMIN
/admin            → ADMIN
```

El reto requiere modificar la entidad, mapear GrantedAuthority en CustomUserDetails y actualizar SecurityFilterChain. No se entrega el código final en esta guía: el estudiante debe diseñar la ampliación a partir de lo aprendido.

21. Reto de ampliación C — Validación del lado servidor

La validación HTML5 mejora la experiencia, pero puede ser evitada por un cliente HTTP. Agregue Jakarta Bean Validation y valide email, nombre, apellido y contraseña en el servidor. Muestre los errores en Thymeleaf y explique por qué la validación del navegador no sustituye la validación del backend.

22. Evidencias de entrega

- Repositorio Git o proyecto Maven completo.
- Captura de la estructura de paquetes.
- Captura de la tabla users y su estructura.
- Evidencia de un registro exitoso.
- Consulta SQL donde se observe la contraseña codificada.
- Evidencia de que /users exige autenticación.
- Evidencia del login exitoso.
- Evidencia del listado de usuarios.
- Evidencia del nombre del usuario autenticado.
- Evidencia del logout.
- Resultado de la prueba UserRepositoryTests.
- Documento breve con respuestas a las preguntas de análisis.
- Implementación de al menos uno de los retos de ampliación.

23. Rúbrica sugerida

Criterio	Peso
Configuración y estructura del proyecto	10%
Persistencia JPA/MySQL y repositorio	15%
Registro y tratamiento seguro de contraseña	20%
Autenticación con Spring Security	20%
Protección de recursos y logout	15%
Vistas Thymeleaf y funcionamiento integral	10%
Pruebas, evidencias y análisis técnico	10%

24. Lista de comprobación final

☐ El proyecto ejecuta con Java 25.

☐ Spring Boot inicia sin errores.

☐ MySQL está conectado.

☐ La tabla users existe.

☐ El correo es único.

☐ El registro funciona.

☐ La contraseña nunca se guarda en texto plano.

☐ El login utiliza el correo electrónico.

☐ /users está protegido.

☐ El nombre del usuario autenticado aparece en la vista.

☐ El logout funciona.

☐ Un usuario no autenticado no puede consultar /users.

☐ La prueba del repositorio finaliza correctamente.

25. Referencias y notas de actualización

Tutorial base: Nam Ha Minh, “Spring Boot Registration and Login with MySQL Database Tutorial - Updated for Spring Boot 3 and Spring Security 6”, CodeJava. El tutorial cubre Spring MVC, Spring Data JPA/Hibernate, Spring Security, Thymeleaf, JUnit 5/AssertJ y MySQL.

Actualización aplicada en esta guía: se utiliza jakarta.persistence, SecurityFilterChain y requestMatchers; se evita WebSecurityConfigurerAdapter y antMatchers. Además, la configuración del DaoAuthenticationProvider usa el constructor que recibe UserDetailsService, acorde con la API actual.

URL de referencia: https://www.codejava.net/frameworks/spring-boot/user-registration-and-login-tutorial

Documentación Spring Boot: https://docs.spring.io/spring-boot/

Documentación Spring Security: https://docs.spring.io/spring-security/reference/